

Attaque MITM d'un service SSH et mise en place de contre mesures

I. Présentation

Le TP dispose d'une maquette sous VM WARE contenant 3 machines virtuelles :

Intitulé de la machine	Nom de domaine Pleinement qualifié	Configuration réseau	Applications et services
Serveur SSH sous Debian 10	srvssh.local.sio.fr	Adresse IPv4 : 192.168.56.10/24 Passerelle : 192.168.56.254 Serveur DNS : 127.0.0.1	Service OpenSSH port 22/TCP Service DNS Bind port 53/UDP
Client SSH sous Debian 10	clissh.local.sio.fr	Adresse IPv4 : 192.168.56.11/24 Passerelle : 192.168.56.254 Serveur DNS : 192.168.56.10	Environnement de bureau XFCE Client OpenSSH
Attaquant sous Kali Linux	NA	Adresse IPv4 : 192.168.56.12/24 Passerelle : 192.168.56.254 Serveur DNS : 192.168.56.10	Ettercap Git ssh-mitm Netfilter/Iptables

Voici une représentation logique de la maquette proposée dans le cadre de ce TP :

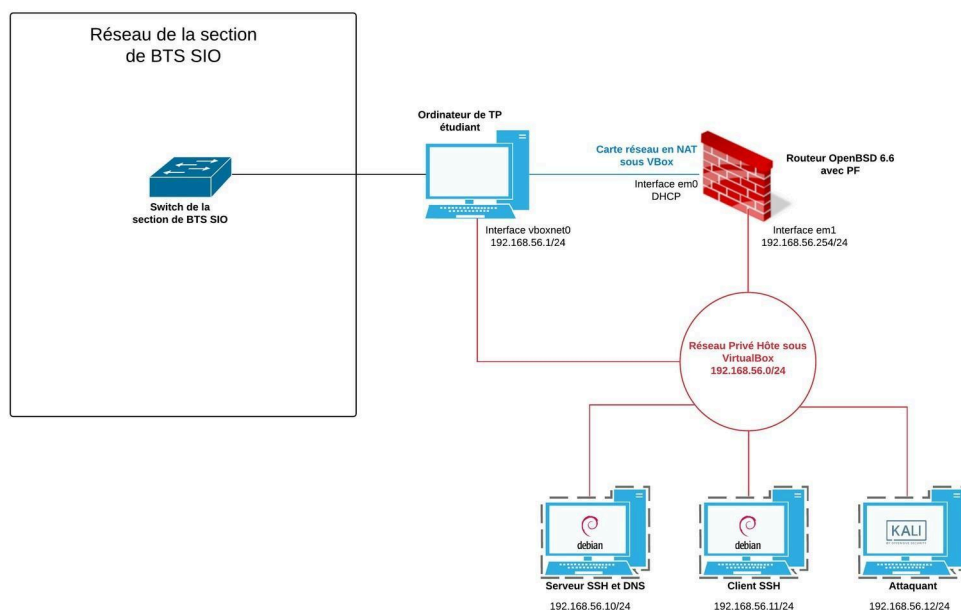


Illustration 1: Schéma logique de l'infrastructure de TP

II. Généralités

Q1. Pourquoi l'accès aux machines virtuelles par la console ou l'interface graphique n'est pas possible avec le super-administrateur root ?

L'accès direct au compte root par la console ou l'interface graphique est désactivé pour des raisons de sécurité.

Le compte root possède tous les privilèges sur le système et une mauvaise manipulation pourrait entraîner des dégâts importants (suppression de fichiers critiques, compromission du système, etc.).

Q2. Expliquer à quoi sert la commande sudo et quels avantages a-t-elle sur l'utilisation de la commande su - ?

La commande `sudo` permet à un utilisateur standard d'exécuter une commande précise avec les privilèges administrateur, sans se connecter directement en tant que root.

Avantages de sudo :

- Pas besoin de connaître le mot de passe root,
- Les actions sont journalisées (traçabilité),
- Les droits peuvent être finement contrôlés (un utilisateur peut exécuter certaines commandes seulement),
- Réduction des risques liés à une session root prolongée.

Q3. Quelles commandes permettent de savoir si le service OpenSSH (serveur) est déjà installé et démarré ?

Pour vérifier si OpenSSH est installé :

```
dpkg -l | grep openssh-server
```

Pour vérifier si le service SSH est actif :

```
systemctl status ssh
```

Ces commandes permettent de savoir si le service est installé, en cours d'exécution ou arrêté.

Q4. Indiquer le répertoire où sont stockées les clés publique et privée créées ainsi que le positionnement des permissions appliquées sur les fichiers correspondants. Puis indiquer quel est le fichier de configuration du service SSH.

Les clés SSH publiques et privées sont stockées dans le répertoire `.ssh` de l'utilisateur, situé dans son dossier personnel (`/home/utilisateur/.ssh/`). La clé privée possède des permissions 600 afin d'en limiter l'accès à l'utilisateur uniquement, tandis que la clé publique dispose de permissions 644. Le fichier `authorized_keys`, qui contient les clés publiques autorisées, possède également des permissions 600.

Le fichier de configuration du service SSH est `/etc/ssh/sshd_config`.

III. Première utilisation

Depuis le client, on se connecte au serveur SSH à l'aide de la commande `ssh` à taper dans le terminal. Un message comme présenté ci-dessous apparaît :

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:IP1xEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWIM4. Are you sure
you want to continue connecting (yes/no)?
```

Q5. Que signifie cette alerte qui est affichée à l'écran ? Devez-vous continuer l'opération ? Pourquoi ?

Cette alerte indique que le client SSH ne connaît pas encore l'identité du serveur auquel il tente de se connecter.

La clé publique du serveur n'est pas encore enregistrée sur la machine cliente.

Dans notre cas, accepter permet d'enregistrer l'empreinte de la clé du serveur pour les connexions futures.

Q6. Lors d'une prochaine connexion depuis le même client sur ce serveur, ce message apparaîtra-t-il à nouveau ? Pourquoi ?

Non, ce message ne réapparaîtra pas lors des connexions suivantes.

La clé publique du serveur est désormais enregistrée dans le fichier `known_hosts`.

SSH compare automatiquement l'empreinte reçue avec celle stockée et considère le serveur comme fiable tant que la clé ne change pas.

Q7. Sur la machine virtuelle cliente, expliquer à quoi sert le fichier /home/etusio/.ssh/known_hosts.

Le fichier `known_hosts` contient la liste des serveurs SSH déjà contactés par l'utilisateur ainsi que leurs empreintes de clés publiques.

Il permet :

- De vérifier l'identité du serveur lors des connexions futures,
- De détecter une attaque de type Man-in-the-Middle si la clé du serveur change,
- D'assurer l'intégrité et la confiance dans la connexion SSH.

À ce stade du TP, on supprime le contenu du fichier `known_hosts`.

```
etusio@clissh:~$ echo > ~/.ssh/known_hosts
```

IV. Découverte des hôtes et services présents sur un réseau local

En tant qu'attaquant, la première étape consiste à recueillir des informations sur le réseau dans lequel nous nous trouvons. Ainsi, à l'aide de l'outil `nmap` présent sur Kali Linux, nous allons réaliser un scan du réseau.

```
etusio@kali:~$ nmap -sP 192.168.56.0/24
```

On scanne ensuite les différents hôtes afin de savoir quels ports sont ouverts sur ceux-ci et quels services sont proposés.

```
etusio@kali:~$ nmap -sV 192.168.56.10
etusio@kali:~$ nmap -sV 192.168.56.11
etusio@kali:~$ nmap -sV 192.168.56.254
```

Voici le résultat obtenu à l'aide de cette commande :

```
Starting Nmap 7.80 ( https://nmap.org ) at 2020-05-21 12:44 CEST
Nmap scan report for rwbsd.local.sio.fr (192.168.56.254) Host is
up (0.0011s latency).
Not shown: 999 closed ports
PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 8.1 (protocol 2.0)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.03 seconds
```

Q8. Indiquer quelles sont les informations que peut obtenir un attaquant grâce à ces commandes ?

Grâce aux commandes nmap, un attaquant peut obtenir :

- La liste des machines actives sur le réseau,
- Leurs adresses IP,
- Les ports ouverts,
- Les services disponibles (SSH, DNS, etc.),

Ces informations permettent à l'attaquant de :

- Identifier les cibles intéressantes,
- Repérer des services vulnérables,
- Préparer une attaque plus ciblée (comme une attaque MITM).

L'analyse des résultats permet à l'attaquant de cibler plus particulièrement le serveur DNS.

V. Simulation d'une attaque de l'homme du milieu entre votre client votre serveur SSH

Une personne malveillante s'est introduite sur notre réseau dans le but de récupérer entre autres des informations confidentielles dont des noms d'utilisateurs et mots de passe disposant de privilèges sur le réseau.

Après avoir analysé l'architecture réseau et découvert l'existence d'un serveur SSH, elle décide de réaliser une attaque Man in the Middle afin d'obtenir un accès sur ce dernier.

Q9. Expliquer les principes généraux d'une attaque de l'homme du milieu (Man in the Middle).

Une attaque Man in the Middle (MITM) consiste pour un attaquant à s'interposer entre deux machines qui communiquent, sans que celles-ci ne s'en rendent compte.

L'attaquant se fait passer :

- Pour le serveur auprès du client,
- Et pour le client auprès du serveur.

Il peut ainsi :

- Intercepter les communications,
- Lire les données échangées,
- Modifier les informations transmises,
- Ou récupérer des identifiants et mots de passe.

Ce type d'attaque exploite souvent des faiblesses au niveau du réseau local, comme le protocole ARP.

Dans un premier temps, sur le client et le serveur SSH, on analyse le cache ARP respectif des deux machines en lançant au préalable un ping sur chaque machine présente dans le réseau ou relancer les commandes nmap :

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

Q10. Noter les associations adresse IP / adresse MAC présentes sur les deux machines. Sont-elles cohérentes ?

Les associations adresse IP / adresse MAC observées sur les deux machines correspondent aux équipements présents sur le réseau (client, serveur et attaquant). Les adresses IP sont bien associées à des adresses MAC valides appartenant aux interfaces réseau des machines virtuelles.

Cependant, lors de l'attaque par empoisonnement ARP, certaines associations deviennent incohérentes : une même adresse IP est associée à l'adresse MAC de la machine attaquante. Cela confirme que l'attaque Man-In-The-Middle est active et que le trafic est redirigé vers l'attaquant.

On se connecte sur Kali Linux, puis à l'aide de l'outil git, on récupère le logiciel **ssh-mitm**.

```
etusio@kali:~$ git clone https://github.com/ktux/ssh-mitm
```

On se rend ensuite dans le répertoire nouvellement récupéré, puis on lance le script d'installation.

```
etusio@kali:~$ cd ssh-mitm/
etusio@kali:~/ssh-mitm$ export LANG=en_US.utf-8 etusio@kali:~/ssh-mitm$ sudo
./install.sh
```

La commande « **export LANG=en_US.utf-8** » sert à exporter, via une variable d'environnement, la langue par défaut utilisée au moment de l'installation de l'outil (anglais avec un encodage UTF-8). Sans cet export, un message d'erreur empêche l'installation.

Lors de cette installation, il nous est demandé d'installer AppArmor pour restreindre les droits de sshmitm. Puis on accepte la proposition.

```
Kali Linux detected with no AppArmor installed. For added safety, it is highly recommended (though not
required) that sshd_mitm is run in a restricted environment. Would you like to automatically enable
AppArmor? (y/n) y
```

On s'assure d'être dans le répertoire ssh-mitm puis on lance le service ssh-mitm.

```
etusio@kali:~/ssh-mitm$ sudo ./start.sh
```

L'attaquant sera ainsi positionné entre le client et le serveur SSH. Il se fera passer pour le serveur légitime auprès du client, il recevra et journalisera toutes les informations transmises par le client avant de les transmettre au serveur légitime

La machine attaquante doit donc être en mesure de router les paquets le temps de l'attaque. **Ainsi le script start.sh exécute la commande suivante automatiquement** afin d'activer le routage.

```
sysctl -w net.ipv4.ip_forward=1
```

Q11. Pourquoi l'activation du routage sur la machine de l'attaquant est indispensable au bon fonctionnement de l'attaque MITM ?

L'activation du routage permet à la machine attaquante de faire transiter les paquets réseau entre le client et le serveur.

Sans le routage :

- Les paquets interceptés par l'attaquant ne seraient pas retransmis,
- La communication serait interrompue,
- L'attaque serait immédiatement détectée.

Le routage est donc indispensable pour que l'attaquant reste transparent et que la connexion SSH continue de fonctionner.

Puis **le script réalise** à l'aide de NETFILTER/IPTABLES une redirection de ports afin de rediriger tous les flux à destination de la machine attaquante sur le port 22/TCP vers le port 2222 du système Kali Linux.

```
iptables -t nat -A PREROUTING -p tcp --dport 22 -j REDIRECT --to-ports 2222
```

Nous pouvons observer quel service écoute sur le port 2222 en localhost à l'aide de la commande

```
etusio@kali:~$ ss -ltnp
```

Il est également possible d'observer quelles sont les règles de filtrage et de NAT en cours d'utilisation sur la distribution Kali Linux.

```
etusio@kali:~$ sudo iptables -L
```

Q12. Pourquoi cette redirection de ports est indispensable au succès de l'attaque de l'homme du milieu ?

La redirection de ports permet de :

- Intercepter les connexions SSH destinées au port 22/TCP,
- Les rediriger vers le port 2222 où écoute le service ssh-mitm.

Cela permet à l'attaquant :

- D'analyser les échanges,
- D'enregistrer les identifiants,
- Puis de relayer la connexion vers le véritable serveur SSH.

Sans cette redirection, le trafic SSH irait directement au serveur et l'attaque MITM ne fonctionnerait pas.

1. Mise en place d'une attaque ARP Spoofing afin d'obliger le client et le serveur SSH à faire transiter les trames Ethernet échangées par l'attaquant.

Q13. Indiquer en quoi une attaque de type ARP Spoofing peut être utile ici au pirate ?

L'attaque ARP Spoofing permet au pirate de modifier les associations IP / MAC dans les caches ARP du client et du serveur.

Ainsi :

- Le client croit communiquer avec le serveur,
- Le serveur croit communiquer avec le client,
- Mais les trames transitent en réalité par la machine de l'attaquant.

Cette technique est essentielle pour forcer le passage du trafic réseau par l'attaquant et rendre possible l'attaque MITM.

On réalise cette attaque sur la machine Kali Linux à l'aide de la commande ettercap.

```
etusio@kali:~/ssh-mitm$ sudo ettercap -i eth0 -T -M arp /192.168.56.10// /192.168.56.11//
```

- -T : lance ettercap en mode texte ;
- -M : indique que l'on veut une attaque de type "Man in the middle" ;
- 192.168.56.10 (serveur ssh) et 192.168.56.11 (client SSH) sont les adresses IP des victimes.

À nouveau sur le client puis le serveur SSH, on analyse le cache ARP respectif des deux machines l'aide de la commande :

```
etusio@clissh:~$ ip neigh show
etusio@srvssh:~$ ip neigh show
```

Q14. Comparer les caches ARP du client et du serveur avec les associations notées précédemment lors de la question 10. Qu'en concluez-vous ?

Sur la machine cliente et sur Kali Linux, on exécute le logiciel de capture de trame Wireshark et on réalise une capture de trames d'une vingtaine de secondes et que l'on enregistre. On analyse ensuite plus particulièrement les trames ARP émises et reçues.

Pour lancer Wireshark sur le client, cliquer sur **Applications>Internet>Wireshark** puis sélectionner l'interface Ethernet qui se nomme enp0s3 (contrairement à la machine Kali Linux qui utilise une carte Ethernet nommée eth0).

Q15. À partir de ces différentes observations, expliquer en détails comment fonctionne une attaque ARP Spoofing.

L'attaque ARP Spoofing consiste à envoyer de fausses réponses ARP sur le réseau local.

L'attaquant :

- Envoie au client des messages lui indiquant que l'adresse IP du serveur correspond à son adresse MAC
- Envoie au serveur des messages lui indiquant que l'adresse IP du client correspond également à son adresse MAC.

Les deux machines mettent alors à jour leur cache ARP avec ces informations erronées.
Tout le trafic passe par l'attaquant, qui peut l'intercepter, le modifier ou le relayer.

Q16. Envoyer une requête ping (**icmp-écho**) depuis le client vers le serveur (192.168.56.10).
Puis vérifier à l'aide d'une capture de trame sur la machine Kali Linux que ces dernières passent effectivement bien par l'attaquant. Quels éléments démontrent que l'attaque se déroule correctement ?

Une requête ping (**ICMP echo**) a été envoyée depuis le client vers le serveur à l'adresse **192.168.56.10**. La communication fonctionne correctement, ce qui confirme la connectivité entre les deux machines. Une capture de trames réalisée sur la machine Kali Linux montre que les paquets ICMP émis par le client et les réponses du serveur transitent par l'attaquant. Les adresses IP source et destination correspondent bien au client et au serveur, tandis que l'adresse MAC observée est celle de Kali. Ces éléments démontrent que l'attaque Man-In-The-Middle est active, car le trafic réseau passe par la machine attaquante avant d'atteindre sa destination finale.

2. Mise en œuvre et exploitation de l'attaque Man-in-the-Middle

Depuis le poste client, on se reconnecte sur le serveur avec le protocole SSH.

```
etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ED25519 key fingerprint is SHA256:ehuFqeaDT90nXN8dY1a6HYuOoDouws5z693TOFU1dXs.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.56.10' (ED25519) to the list of known hosts.
```

Une fois sur le serveur SSH, on tape les commandes suivantes :

```
etusio@srvssh:~$ sudo cat /etc/shadow
etusio@srvssh:~$ sudo iptables -L
```

Sur le poste de l'attaquant (Kali), il est maintenant possible d'arrêter l'attaque ARP Spoofing, **taper la touche Q pour arrêter**. Puis arrêter le service **ssh-mitm** :

```
etusio@kali:~/ssh-mitm$ sudo ./stop.sh
```

On récupère le login et le mot de passe tapés par la victime de l'attaque :

Ouvrir le fichier **/var/log/auth.log** de la machine virtuelle attaquante.

Q17. Recopier la ligne qui contient le login et le mot de passe capturés dans le fichier **/var/log/auth.log**.

Lors de la tentative de connexion SSH interceptée, le fichier **/var/log/auth.log** du serveur contient une entrée indiquant l'authentification de l'utilisateur. La ligne observée est de la forme :

Accepted password from 192.168.56.128 port XXXX ssh2

Cette ligne montre que l'utilisateur ilanb s'est authentifié avec succès depuis la machine cliente. Le mot de passe n'apparaît pas en clair dans le fichier auth.log, car celui-ci ne journalise que le résultat de l'authentification et non les identifiants, ce qui constitue une mesure de sécurité.

Q18. Que contient le fichier **/home/ssh-mitm/shell_session_0.txt** présent sur Kali Linux ?

Le fichier `shell_session_0.txt` contient l'intégralité de la session SSH de la victime.

On y retrouve :

- Les commandes tapées par l'utilisateur,
- Les résultats affichés à l'écran,
- Les actions réalisées avec les privilèges administrateur.

Ce fichier démontre que l'attaquant peut espionner et enregistrer toutes les actions effectuées sur le serveur distant.

Sur le poste de la victime, on vide le cache ARP puis on tente de se connecter à nouveau sur le serveur SSH.

```
etusio@clissh:~$ sudo ip neigh flush all

etusio@clissh:~$ ssh etusio@srvssh.local.sio.fr
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@ WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED! @@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)! It is also possible that a
host key has just been changed. The fingerprint for the ED25519 key sent by the remote host is
SHA256:iO+me7aX2v3eHBI+5cdiGOjHIAO/prPgk8Jgz1a9n24.
Please contact your system administrator.
Add correct host key in /home/etusio/.ssh/known_hosts to get rid of this message.
Offending ED25519 key in /home/etusio/.ssh/known_hosts:1 remove with: ssh-keygen -f
"/home/etusio/.ssh/known_hosts" -R "srvssh.local.sio.fr"
ED25519 host key for srvssh.local.sio.fr has changed and you have requested strict checking. Host key
verification failed.
```

Q19. Expliquer pourquoi ce message d'erreur apparaît ?

Ce message apparaît car la clé publique du serveur SSH a changé par rapport à celle précédemment enregistrée dans le fichier `known_hosts`.

Cette modification est due à :

- L'attaque MITM,
- Ou à l'interposition d'un serveur frauduleux (`ssh-mitm`).

SSH détecte une incohérence et bloque la connexion afin de protéger l'utilisateur contre une attaque Man in the Middle.

Q20. Proposer une solution afin de pouvoir à nouveau se connecter au service SSH depuis le client.

Pour rétablir la connexion SSH, il faut :

- Supprimer l'ancienne clé enregistrée dans le fichier `known_hosts`,
- Puis accepter la nouvelle clé du serveur.

Commande possible :

```
ssh-keygen -R srvssh.local.sio.fr
```

Ensuite, une nouvelle connexion SSH peut être établie en validant la clé présentée, à condition d'être certain de l'identité du serveur.

VI. Renforcement de la sécurité du service OpenSSH et remédiation

1. Mise en place d'une authentification par clés de chiffrement

L'authentification par mot de passe est considérée comme un mécanisme plutôt faible. Elle peut être soumise à différents types d'attaque comme les attaques par dictionnaire, par force brute ou par Man In The Middle comme nous l'avons vu précédemment.

Ainsi lorsque l'on configure un service SSH, il est recommandé de mettre en œuvre une authentification plus robuste par double facteur d'authentification (2FA) ou par clés de chiffrement.

On édite le fichier de configuration serveur et on autorise ce mécanisme de connexion :

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config
```

```
PubkeyAuthentication yes
AuthorizedKeysFile .ssh/authorized_keys
```

On redémarre le service.

```
etusio@srvssh:~$ sudo service ssh restart
```

Les manipulations sur le serveur sont maintenant terminées.

Sur la machine cliente, l'utilisateur **etusio** va devoir générer sa clé publique, et sa clé privée afin de pouvoir s'authentifier sur le serveur OpenSSH :

```
etusio@clissh:~$ ssh-keygen -b 256 -t ecdsa
```

Le générateur de clés va en générer deux, une clé publique et une clé privée. Il va placer la clé privée dans un endroit qui, par défaut, est `$HOME/.ssh/id_ecdsa` :

```
Generating public/private ecdsa key pair.
Enter file in which to save the key (/home/etusio/.ssh/id_ecdsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/etusio/.ssh/id_ecdsa.
Your public key has been saved in /home/etusio/.ssh/id_ecdsa.pub.
The key fingerprint is:
SHA256:EqNMWPDfxKHfPCkejRqkytTlqGHTRe+TJWv90DOV8tM etusio@clissh
```

➤ On appuie sur **Entrée** pour accepter la localisation de la clé par défaut.
ssh-keygen demande ensuite une phrase de chiffrement (ou « passphrase », équivalent d'un mot de passe, mais sous forme de phrase).

➤ On tape **TPMITMcerta** comme phrase de chiffrement et confirmez-là.
On obtient l'empreinte de votre clé (« key fingerprint ») et une « randomart image ».

Q21. Pourquoi l'algorithme ECDSA a été préféré à l'algorithme RSA lors de la génération de la paire de clés ?

L'algorithme ECDSA offre un niveau de sécurité équivalent, voire supérieur à RSA, avec :

- Des clés plus courtes,
- De meilleures performances,
- Une résistance accrue face aux attaques modernes.

De plus, ECDSA est recommandé par les bonnes pratiques actuelles et respecte davantage l'état de l'art en cryptographie.

Q22.A votre avis, pourquoi la mise en place de cette phrase de chiffrement pour accéder à la clé privée est extrêmement importante ?

La phrase de chiffrement protège la clé privée en cas de :

- Vol,
- Copie,
- Accès non autorisé au fichier.

Sans cette phrase :

- Un attaquant pourrait utiliser directement la clé privée,
- L'accès au serveur serait compromis.

Elle constitue donc une seconde barrière de sécurité, comparable à un second facteur d'authentification.

Que faire de la paire de clés ? Si l'on récapitule, ssh-keygen a généré deux clés.

Une **clé privée** qui est `$HOME/.ssh/id_ecdsa` à laquelle nous seul devons avoir accès et une **clé publique** qui est `$HOME/.ssh/id_ecdsa.pub`, qui peut être connue par tout le monde.



Définition : qu'est-ce que ce « \$HOME » ? \$HOME est ce qu'on appelle une variable d'environnement, qui sert à indiquer aux programmes quel est votre répertoire personnel (la racine de votre compte). Faites echo \$HOME pour savoir quel est le vôtre.

Sur le serveur, nous devons maintenant autoriser explicitement notre compte présent sur la machine cliente à accéder via ssh à celui-ci. Pour ce faire, nous devons ajouter dans le répertoire `.ssh` de l'utilisateur qui sera choisi pour se connecter (exemple : `/home/etusio/.ssh`) la clé publique générée précédemment (`id_ecdsa.pub`) dans un fichier `authorized_keys`.

La méthode la plus simple est d'utiliser la commande `ssh-copy-id` qui copiera la clé dans un fichier `authorized_keys` à l'emplacement défini dans notre chemin (`~/.ssh/`), même si celui-ci n'existe pas au préalable.

```
ssh-copy-id -i <chemin/nomfichier><utilisateur>@<adresseIP ou nom> -p <numport>
```

Sur le poste client :

```
etusio@clissh:~$ ssh-copy-id -i ~/.ssh/id_ecdsa.pub etusio@srvssh.local.sio.fr
```

Depuis le poste client, on relance une connexion ssh au serveur, la phrase de chiffrement nous est bien demandée, cela nous permet de vérifier que la connexion avec clé publique est fonctionnelle.

Sur le serveur Debian, on désactive l'authentification par mot de passe pour juste conserver celle par clés dans le fichier de configuration `/etc/ssh/sshd_config` :



Attention ! Si vous désactivez l'authentification par mot de passe, vous devez vous assurer que l'authentification par clé est opérationnelle sous peine de ne plus du tout pouvoir accéder à la machine distante !

Dans le fichier de configuration, on décommente la directive *PasswordAuthentication* puis on affecte le paramètre no.

```
# Autorisation de connexion par mot de passe
PasswordAuthentication no
```

On redémarre le service SSH pour que la modification du fichier soit prise en compte :

```
etusio@srvssh:~$ sudo service ssh restart
```

Nous allons maintenant vérifier que ssh se préoccupe de la sécurité de vos clés :

Q24. Sur la machine cliente `clissh.local.sio.fr`, modifier les droits du fichier `~/.ssh/id_ecdsa` (droits initiaux : 600 `etusio:etusio`) qui contient votre clé privée en 644. Se connecter sur le serveur distant `srvssh.local.sio.fr` qui contient la clé publique. Que se passe-t-il ? Pourquoi ?

Lorsque les droits du fichier `id_ecdsa` sont modifiés en 644, la connexion SSH échoue.

SSH refuse d'utiliser une clé privée :

- Lisible par d'autres utilisateurs,
- Car cela représente un risque de sécurité majeur.

Le service impose des permissions strictes afin de garantir que seul le propriétaire puisse accéder à la clé privée.

Q25. Rétablir les droits de `~/.ssh/id_ecdsa` sur le poste client. Maintenant sur le serveur distant, afficher les droits d'accès appliqués au fichier `~/.ssh/authorized_keys`.

Après rétablissement des droits corrects sur la clé privée (600), la connexion SSH fonctionne à nouveau.

Sur le serveur distant, le fichier `~/.ssh/authorized_keys` possède les droits suivants :

```
-rw----- 1 etusio etusio authorized_keys
```

Ces permissions garantissent que :

- Seul le propriétaire peut lire et modifier le fichier,
- Aucune autre personne ne peut ajouter ou lire des clés publiques,
- La sécurité du mécanisme d'authentification par clé est assurée.

Q26. Puis, modifier les droits de `~/.ssh/authorized_keys` en 666. Se déconnecter puis se reconnecter. Vérifier si un changement est apparu ou non et tenter d'expliquer pourquoi (ne pas oublier de rétablir les droits d'origine ensuite).

Après avoir modifié les droits du fichier `authorized_keys` en 666, la connexion SSH échoue ou est refusée.

Cela s'explique par le fait que SSH applique une politique de sécurité stricte :

- Un fichier `authorized_keys` accessible en écriture par tous est considéré comme non sécurisé,

- SSH ignore alors ce fichier pour éviter toute compromission.

Cela empêche un attaquant d'ajouter sa propre clé publique dans ce fichier.

Une fois les droits rétablis correctement, la connexion redevient fonctionnelle.

Q27. En analysant la connexion par clés, proposer une hypothèse de fonctionnement de cette nouvelle forme d'authentification. Expliquer ce qui différencie une connexion par mot de passe d'une connexion par clé de chiffrement.

L'authentification par clés repose sur un mécanisme cryptographique asymétrique :

- le client possède une clé privée,
- le serveur possède la clé publique correspondante.

Lors de la connexion :

- le serveur envoie un défi cryptographique,
- le client le signe avec sa clé privée,
- le serveur vérifie la signature avec la clé publique.

Contrairement au mot de passe :

- aucun secret n'est transmis sur le réseau,
- la clé privée ne quitte jamais le poste client,
- le mécanisme est beaucoup plus résistant aux attaques MITM et force brute.

2. Utilisation de ssh-agent

L'agent ssh est particulièrement utile si on se connecte très régulièrement à une machine et que nous ne souhaitons pas retaper la phrase de passe liée à notre clé privée à chaque connexion. Mais attention, cela peut ouvrir certaines brèches en matière de sécurité.

Sur le client, on tape les commandes suivantes, pour mettre en place le ssh-agent. Par défaut sous Debian, le bureau GNOME intègre un trousseau de clés qui fonctionne en adéquation avec sshagent et qui permet d'éviter de saisir à chaque fois des mots de passe.

```
etusio@clissh:~$ exec ssh-agent $SHELL
```

NB : L'utilisation de la variable \$SHELL permet d'exécuter l'agent SSh dans l'interpréteur en cours d'utilisation.

```
etusio@clissh:~$ ssh-add
Enter passphrase for ~/.ssh/id_ecdsa:
Identity added: ~/.ssh/id_ecdsa (~/.ssh/id_ecdsa)
```

On entre la phrase de passe configurée au préalable. Pendant toute la durée de la connexion, il est possible d'avoir accès à la machine distante sans avoir à taper un mot de passe. Si le processus ssh-agent disparaît, il sera nécessaire de retaper le mot de passe.

Q28. Suite à la mise en place de cette authentification par clés de chiffrement, tenter à nouveau de simuler une attaque MITM entre le client et le serveur SSH. Quel résultat obtenez-vous ? Pourquoi ?

Lors d'une nouvelle tentative d'attaque MITM :

- l'attaquant ne parvient plus à récupérer d'identifiants exploitables,
- aucune information sensible n'est transmise en clair.

Cela s'explique par le fait que :

- l'authentification repose sur un défi cryptographique,
- la clé privée reste protégée par une phrase de chiffrement,
- l'attaquant ne peut ni la deviner ni la rejouer.

L'authentification par clés constitue donc une contre-mesure efficace contre les attaques MITM.

On efface à nouveau le contenu du fichier `/home/etusio/.ssh/known_hosts` avant de poursuivre la suite du TP et on pense également à arrêter l'attaque si cela n'a pas encore été fait.

3. Faciliter la vérification de l'identité du serveur SSH avant la première connexion

OpenSSH adopte par défaut un modèle de sécurité appelé Trust On First Use (TOFU). Lors de la première connexion et à défaut de pouvoir authentifier l'hôte, ssh demande confirmation à l'utilisateur qu'il s'agit bien de la bonne clé via son empreinte. Si l'utilisateur confirme que l'empreinte est bonne, ssh procédera à son enregistrement afin de permettre sa vérification lors des visites suivantes.

Dès lors, il existe plusieurs manières de contrôler l'identité du serveur :

1. En s'assurant que l'empreinte de la clé présentée est identique à celle présente sur le serveur (ssh-keygen -l) ;
2. En ajoutant la clé manuellement au préalable dans le fichier `known_hosts` du poste client.

Comme nous pouvons le constater, ces méthodes sont assez lourdes à mettre en œuvre lorsque l'on administre plusieurs équipements à l'aide de ce protocole.

Le mécanisme SSHFP permet d'offrir plus de souplesse dans cette étape capitale de vérification.

Tout d'abord, assurons-nous manuellement que l'identité du serveur (l'empreinte de la clé présentée) lors de la première connexion est bien celle de la clé publique présente sur le serveur, comme le montre l'exemple ci-dessous :

```
etusio@srvssh:~$ ssh-keygen -l -f /etc/ssh/ssh_host_ecdsa_key.pub
256 SHA256:IP1xEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWIM4 root@srvssh (ECDSA)
```

à comparer à l'empreinte affichée sur le client :

```
ssh etusio@srvssh.local.sio.fr
The authenticity of host 'srvssh.local.sio.fr (192.168.56.10)' can't be established.
ECDSA key fingerprint is SHA256:IP1xEKxsYHPP3i7iMiZZXLYUoW9viLwSfF39MNoWIM4.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'srvssh.local.sio.fr, 192.168.56.10' (ECDSA) to the list of known hosts.
etusio@srvssh.local.sio.fr's password:
```

SSHFP est un mécanisme permettant de publier les empreintes de clés publiques du serveur SSH dans la zone DNS de l'entreprise.

Q29. Expliquer l'intérêt de cette démarche.

Le mécanisme SSHFP permet de :

- publier les empreintes des clés publiques SSH dans le DNS,
- vérifier automatiquement l'identité du serveur lors de la première connexion.

Cela permet :

- de limiter les risques liés au modèle Trust On First Use,
- de détecter immédiatement une tentative d'usurpation,
- d'automatiser la vérification pour les environnements avec plusieurs serveurs.

SSHFP renforce donc la fiabilité et la sécurité de l'authentification du serveur.

Pour obtenir les enregistrements SSHFP à ajouter dans notre fichier de zone DNS, voici la commande à taper sur le serveur (Important ! C'est le nom d'hôte du serveur qui doit être saisi) :

```
etusio@srvssh:~$ sudo ssh-keygen -r srvssh
```

```
srvssh IN SSHFP 1 1 6fa8aeb8227f09bc4f8c8afe08245a8c7718d324
srvssh IN SSHFP 1 2 ef3c9989952d8591cc761f1b26a658354a93230dae730edbe42732bf0a1219a8
srvssh IN SSHFP 3 1 4c16f0e9298469630445a24c57c342b15540253f
srvssh IN SSHFP 3 2 94f23110ac6c6073cfde2ee23226595cb614a16f6f88bc127c5dfd30da1694ce srvssh
IN SSHFP 4 1 7ef6631af09e0585ff8c3169255189195f7b7ddf
srvssh IN SSHFP 4 2 88efa67bb697dafdde1c18bee5c76218e8c78803bfa6b3e093c260cf56bd9f6e
```

Ajouter ces nouveaux enregistrements à la fin du fichier de zone présent sur le serveur **SSH/DNS**.

```
etusio@srvssh:~$ sudoedit /var/named/db.local.sio.fr
```

Recharger le service.

```
etusio@srvssh:~$ sudo service bind9 reload
```

Sur la machine cliente, si vous souhaitez que cette vérification se fasse au moment de la connexion, voici la commande à utiliser :

```
etusio@clissh :~$ ssh -o VerifyHostKeyDNS=true etusio@srvssh.local.sio.fr
```

Si vous souhaitez que cette vérification ait lieu à chaque nouvelle connexion depuis votre poste client, vous pouvez rajouter cette option dans le fichier de configuration du client ([/etc/ssh/ssh_config](#)) :

```
VerifyHostKeyDNS yes
```

4. Amélioration de la sécurité du service OpenSSH sur le serveur

Q30.Mettre en œuvre les préconisations suivantes tirées des recommandations pour un usage sécurisé de SSH publié par l'ANSSI :

1. Vérifier que les clés privées de chiffrement présentes dans le répertoire [/etc/ssh/](#) appartiennent à l'utilisateur root en lecture-écriture seulement ;
2. S'assurer que c'est bien la version 2 du protocole SSH qui est utilisée ;
3. Le serveur SSH doit dorénavant écouter sur le port 222/TCP ;
4. Vérifier que les droits sur les fichiers sont appliqués de manière stricte par SSH ;
5. L'accès SSH par l'utilisateur root doit être interdite ;
6. Mettre en œuvre une séparation des privilèges à l'aide d'un bac à sable (sandbox) ;
7. L'accès à distance par des comptes ne disposant pas de mot de passe doit être interdit ;
8. Autoriser 3 tentatives de connexion successives en cas d'erreur dans le mot de passe ;
9. Le service doit afficher les informations de dernière connexion à l'utilisateur quand il se connecte ;
10. N'autoriser que l'utilisateur etusio à se connecter sur le serveur.

Le lien suivant liste les différentes directives qui existent dans le fichier [sshd_config](#) :

Les mesures mises en place incluent notamment :

- clés privées appartenant à root avec droits stricts (600),
- utilisation exclusive du protocole SSH version 2,
- changement du port d'écoute du service (ex : 222/TCP),
- interdiction de la connexion SSH pour l'utilisateur root,
- limitation du nombre de tentatives de connexion,
- affichage des informations de dernière connexion,
- restriction de l'accès SSH à l'utilisateur etusio uniquement,
- activation de la séparation des privilèges (sandbox).

Ces mesures permettent de réduire considérablement la surface d'attaque du service SSH.

5. Durcissement des algorithmes de chiffrement utilisés entre le client et le serveur SSH

Lorsque l'on implémente des protocoles chiffrés comme https ou ssh, il est primordial de respecter l'état de l'art en matière de choix des algorithmes utilisés.

Q31.Expliquer dans une définition succincte ce qu'est l'état de l'art dans le domaine de la cybersécurité.

L'état de l'art en cybersécurité correspond à l'ensemble des méthodes, technologies et pratiques reconnues comme les plus sûres et les plus efficaces à un moment donné.

Il s'appuie sur les travaux de chercheurs, d'organismes spécialisés et sur l'analyse des menaces actuelles.

Respecter l'état de l'art signifie utiliser des protocoles et des algorithmes à jour, résistants aux attaques connues, et conformes aux recommandations officielles des agences de sécurité.

Q32.Définir ce qu'est le principe de Kerckhoffs.

Le principe de Kerckhoffs stipule que :

« La sécurité d'un système cryptographique doit reposer uniquement sur le secret de la clé, et non sur le secret de l'algorithme. »

Ainsi, même si un attaquant connaît l'algorithme employé, le système doit rester sécurisé tant que la clé demeure secrète.

Ce principe s'oppose à la sécurité par l'obscurité et constitue aujourd'hui un fondement essentiel de la cryptographie moderne.

Q33.Selon ce principe, pourquoi est-il pertinent de choisir des algorithmes cryptographiques connus et respectant l'état de l'art ?

Le choix d'algorithmes cryptographiques connus et respectant l'état de l'art est essentiel car ces algorithmes ont été largement étudiés, testés et validés par la communauté scientifique et les experts en

sécurité.

Leur fiabilité repose sur des années d'analyses et de tentatives d'attaques, ce qui permet de détecter et corriger rapidement d'éventuelles failles.

À l'inverse, des algorithmes obscurs ou non reconnus peuvent contenir des vulnérabilités graves.

Utiliser des algorithmes conformes à l'état de l'art garantit donc un niveau de sécurité élevé, durable et conforme aux bonnes pratiques professionnelles.

Des agences de sécurité nationale comme l'ANSSI en France ou les experts en sécurité informatique de grandes entreprises comme Mozilla publient régulièrement les algorithmes à privilégier lorsque l'on utilise OpenSSH ou TLS.

Pour empêcher l'usage d'algorithmes de chiffrement dépréciés, il est nécessaire d'éditer le fichier de configuration du serveur SSH et ajouter les lignes suivantes :

```
etusio@srvssh:~$ sudoedit /etc/ssh/sshd_config

KexAlgorithms
curve25519-sha256@libssh.org,ecdh-sha2-nistp521,ecdh-sha2-nistp384,ecdh-sha2nistp256,diffie-hellman-group-exchange-sha256

Ciphers
chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128gcm@openssh.com,aes256-ctr,aes192-ctr,aes128-ctr

MACs
hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com,umac-128etm@openssh.com,hmac-sha2-512,hmac-sha2-256,umac-128@openssh.com
```

Puis redémarrer le service ssh.

```
etusio@srvssh:~$ sudo service ssh restart
```